

Java OOP Practical Quiz

Quiz 02: Inheritance and Polymorphism - Method Overloading and Method Overriding

Instructions

- Answer all questions. Manage your time carefully.
- Use class name Main for executable programs unless another name is clearly requested.
- Do not use exception handling unless you already know it. Focus on correct OOP logic.
- For coding questions, use clear variable names and readable output messages.
- Marks are given for correct logic, correct OOP concept usage, and clean code structure.

Part A

Answer the following five questions. Each question carries 3 marks. Write short but clear answers.

Question 1 (3 marks)

```
class Printer {  
    void print(int x) {  
        System.out.println(x);  
    }  
  
    int print(int y) {  
        return y;  
    }  
}
```

Is this valid method overloading? Explain your answer.

Question 2 (3 marks)

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Meow");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Cat();  
        a.sound();  
    }  
}
```

What is the output? Explain why this output is printed.

Question 3 (3 marks)

```
class Calculator {  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    double add(double a, double b) {  
        return a + b;  
    }  
}  
  
class AdvancedCalculator extends Calculator {  
    int add(int a, int b) {  
        return a + b + 10;  
    }  
}
```

Identify one example of method overloading and one example of method overriding from this code.

Question 4 (3 marks)

```
class Employee {
    void work() {
        System.out.println("Employee works");
    }
}

class Manager extends Employee {
    void work() {
        System.out.println("Manager manages work");
    }

    void approveLeave() {
        System.out.println("Leave approved");
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Manager();
        e.work();
        e.approveLeave();
    }
}
```

Will this program run successfully? If not, identify the error and explain why.

Question 5 (3 marks)

```
class Message {
    void show(String text) {
        System.out.println(text);
    }

    void show(int number) {
        System.out.println(number);
    }

    void show(String text, int number) {
        System.out.println(text + " " + number);
    }
}
```

For each method call below, write which method will be executed. If the call is invalid, write invalid and give the reason.

```
show("Java");
show(25);
show("OOP", 10);
show(10, "OOP");
```

Part B

Employee Payment System with Overloading and Overriding (15 marks)

Create a Java program for an employee payment system using inheritance, method overriding, method overloading, and polymorphism.

Requirements

Create a parent class named Employee with the following data members:

```
employeeId  
employeeName
```

Create a constructor to initialize these values.

Create the following method in the Employee class:

```
double calculatePayment()
```

In the parent class, this method can return 0.

Create the following overloaded methods in the Employee class:

```
void displayDetails()  
void displayDetails(String companyName)  
void displayDetails(String companyName, double payment)
```

- displayDetails() should display only employee ID and employee name.
- displayDetails(String companyName) should display company name, employee ID, and employee name.
- displayDetails(String companyName, double payment) should display company name, employee ID, employee name, and payment.

Child Class 1 - FullTimeEmployee

- Create a child class named FullTimeEmployee.
- Add a data member named monthlySalary.
- Override calculatePayment() to return the monthly salary.

Child Class 2 - PartTimeEmployee

- Create a child class named PartTimeEmployee.
- Add data members named hoursWorked and hourlyRate.
- Override calculatePayment() to return hoursWorked * hourlyRate.

Main Class Requirement

- Create an Employee[] array with 3 employees.
- The array must contain one FullTimeEmployee and two PartTimeEmployee objects.
- Use a loop to display each employee details and payment.
- Inside the loop, call the overloaded displayDetails methods and calculatePayment method.

Your main method must use a structure similar to this:

```
Employee[] employees = new Employee[3];  
  
employees[0] = new FullTimeEmployee(...);  
employees[1] = new PartTimeEmployee(...);  
employees[2] = new PartTimeEmployee(...);  
  
for (int i = 0; i < employees.length; i++) {  
    employees[i].displayDetails();  
    employees[i].displayDetails("ABC Company");  
    employees[i].displayDetails("ABC Company", employees[i].calculatePayment());  
}
```

}

Marking Guide for Part B

Criteria	Marks
Correct Employee parent class with constructor and fields	3
Correct FullTimeEmployee and PartTimeEmployee child classes	4
Correct method overriding of calculatePayment()	3
Correct method overloading of displayDetails()	3
Correct polymorphic Employee[] array, loop, and readable output	2
Total	15

Student Checklist

- I used inheritance correctly.
- I used method overriding correctly.
- I used method overloading correctly.
- I used an Employee[] array to store child class objects.
- I tested the output before submitting.